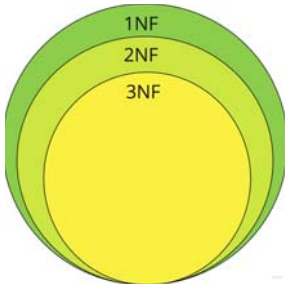


## A 3.2.6 Database Scenarios



### A3.2.6 Constructing databases

Learn to construct 3NF databases for real-world scenarios (e.g., library, hospital, e-commerce, school, employee, inventory, crime reporting). Design efficient, normalized databases for practical applications.

### Library Management System



#### Initial Unnormalized Design (Conceptual):

A single table with information about books, authors, members, and loans. This leads to much redundancy.

#### 3NF Normalized Design:

##### Authors table:

**AuthorID** (PK), **FirstName**, **LastName**, **Nationality**

##### Books table:

**BookID** (PK), **Title**, **ISBN**, **PublicationYear**, **AuthorID** (FK)

##### Members table:

**MemberID (PK), FirstName, LastName, Address, Phone**

#### Loans table:

**LoanID (PK), BookID (FK), MemberID (FK), LoanDate, DueDate, ReturnDate**

### Key Normalization Points:

- Separate tables for Authors, Books, and Members to reduce redundancy.
- Foreign keys (AuthorID, BookID, MemberID) to establish relationships.
- Transitive dependency removed: For example. the Author's nationality is in the Author table, not repeated in Books.

## Hospital Management System



### Initial Unnormalized Design (Conceptual):

A table combining patient, doctor, appointment, and treatment details.

### 3NF Normalized Design:

#### Patients table:

- **PatientID (PK), FirstName, LastName, DateOfBirth, Address, Phone**

#### Doctors table:

- **DoctorID (PK), FirstName, LastName, Specialization, Department**

#### Appointments table:

- AppointmentID (PK), PatientID (FK), DoctorID (FK), AppointmentDate, AppointmentTime, RoomNumber

#### Treatments table:

- TreatmentID (PK), TreatmentName, Description

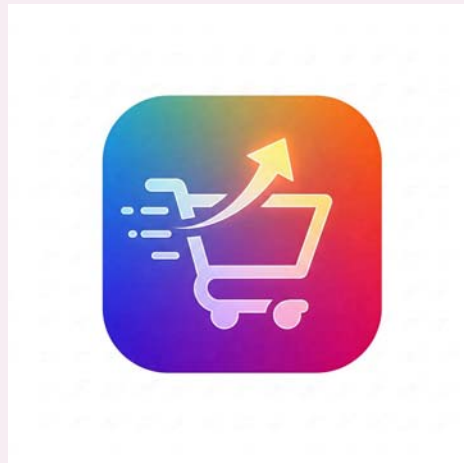
#### PatientTreatments table: (Linking table for many-to-many relationship)

- PatientID (FK), TreatmentID (FK), DateGiven, Dosage (Composite PK)

#### Key Normalization Points:

- Separate tables for Patients, Doctors, Appointments, Treatments.
- Foreign keys to link appointments to patients and doctors.
- A linking table (PatientTreatments) to handle the fact that a patient can have many treatments, and a treatment can be given to many patients.
- Transitive dependency removed: Doctor's department is in the Doctors table.
- Address is left in the table for simplicity in a fully atomic table this would be broken down further.

## E-commerce Platform



#### Initial Unnormalized Design (Conceptual):

- A table with order, customer, product, and shipping information.

## 3NF Normalized Design

#### Customers table:

**CustomerID (PK), FirstName, LastName, Email, ShippingAddress, BillingAddress**

### **Products table:**

**ProductID (PK), ProductName, Description, Price, CategoryID (FK)**

### **Orders table:**

**OrderID (PK), CustomerID (FK), OrderDate, OrderStatus**

### **OrderItems table: (Linking table)**

**OrderID (FK), ProductID (FK), Quantity, UnitPrice (Composite PK)**

### **Categories table:**

**CategoryID (PK), CategoryName**

## **Key Normalization Points**

- Separate tables for Customers, Products, Orders.
- OrderItems table to handle multiple products in an order.
- Foreign keys to link orders to customers and order items to orders and products.
- Transitive dependency removed: Product category name is in the Categories table.

## **School Management System**



### **Initial Unnormalized Design (Conceptual):**

A table with student, course, teacher, and grade information.

## 3NF Normalized Design:

### Students table:

**StudentID** (PK), **FirstName**, **LastName**, **DateOfBirth**, **Major**

### Courses table:

**CourseID** (PK), **CourseName**, **Credits**, **DepartmentID** (FK)

### Teachers table:

**TeacherID** (PK), **FirstName**, **LastName**, **HireDate**, **DepartmentID** (FK)

### Enrollments table: (Linking table)

**StudentID** (FK), **CourseID** (FK), **Grade**, **Semester** (Composite PK)

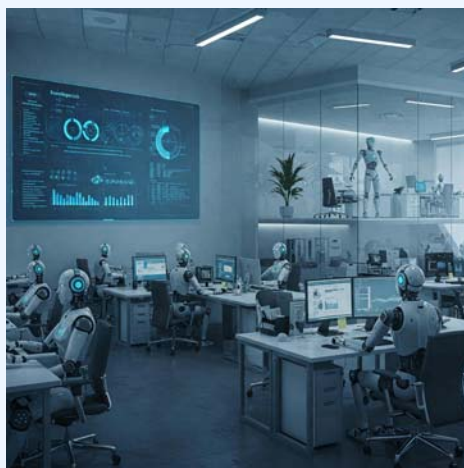
### Departments table:

**DepartmentID** (PK), **DepartmentName**

## Key Normalization Points:

- Separate tables for Students, Courses, Teachers.
- Enrollments table for student-course relationships.
- Foreign keys to link courses and teachers to departments.
- Transitive dependency removed: Department name is in the Departments table.

## Employee Management System



## Initial Unnormalized Design (Conceptual):

A table with employee, department, and project assignment details.

## 3NF Normalized Design:

### Employees table:

EmployeeID (PK), FirstName, LastName, HireDate, Salary, DepartmentID (FK)

### Departments table:

DepartmentID (PK), DepartmentName, Location

### Projects table:

ProjectID (PK), ProjectName, StartDate, EndDate

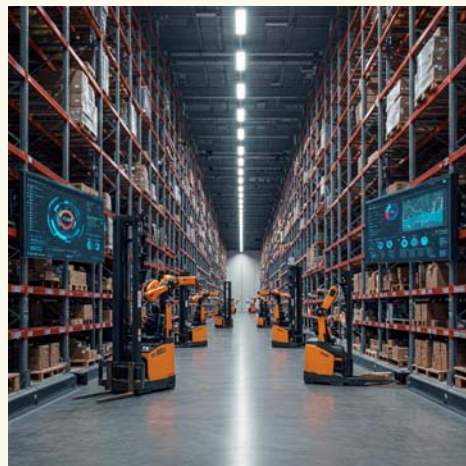
### Assignments table: (Linking table)

EmployeeID (FK), ProjectID (FK), Role, HoursWorked (Composite PK)

## Key Normalization Points:

- Separate tables for Employees, Departments, Projects.
- Assignments table to manage employee-project relationships.
- Foreign keys to link employees to departments.
- Transitive dependency removed: Department location is in the Departments table.

## Inventory Management System



## Initial Unnormalized Design (Conceptual):

A table with product, supplier, and order information.

## 3NF Normalized Design:

### Products table:

ProductID (PK), ProductName, Description, UnitPrice, SupplierID (FK)

### Suppliers table:

SupplierID (PK), SupplierName, Address, ContactPerson

### Orders table:

OrderID (PK), OrderDate, CustomerID (FK)

### OrderItems table: (Linking table)

OrderID (FK), ProductID (FK), QuantityOrdered (Composite PK)

### Customers table:

CustomerID (PK), CustomerName, CustomerAddress

### Key Normalization Points:

- Separate tables for Products, Suppliers, Orders.
- OrderItems table to manage products within orders.
- Foreign keys to link products to suppliers and orders to customers.
- Transitive dependency removed: Supplier address is in the Suppliers table.

## Police Crime Reporting System



## Initial Unnormalized Design (Conceptual):

A table with crime, location, officer, and suspect information.

## 3NF Normalized Design:

### Crimes table:

**CrimeID** (PK), **CrimeType**, **CrimeDate**, **CrimeTime**, **LocationID** (FK)

### Locations table:

**LocationID** (PK), **StreetAddress**, **City**, **State**, **ZipCode**

### Officers table:

**OfficerID** (PK), **FirstName**, **LastName**, **BadgeNumber**, **PrecinctID** (FK)

### Suspects table:

**SuspectID** (PK), **FirstName**, **LastName**, **DateOfBirth**, **Address**

### Involved table: (Linking table)

**CrimeID** (FK), **SuspectID** (FK), **OfficerID** (FK), **Role** (Composite PK)

### Precincts table:

**PrecinctID** (PK), **PrecinctName**, **PrecinctAddress**

### Key Normalization Points:

- Separate tables for Crimes, Locations, Officers, Suspects.
- Involved table to link crimes to suspects and officers.
- Foreign keys to establish relationships (e.g., crimes to locations, officers to precincts).



- Transitive dependency removed: Location details are in the Locations table, precinct details are in the Precincts table.

## Quiz

Now test your knowledge by trying this quiz.

1

Fill in the blanks with the most appropriate word or phrase from the list provided below.

|                       |            |               |                       |                       |                |  |
|-----------------------|------------|---------------|-----------------------|-----------------------|----------------|--|
|                       |            | relationships | 2NF                   | transitive dependency | OrderItems     |  |
| composite primary key | Orders     | Customers     | partial dependencies  | atomic                | separate       |  |
| many-to-many          | 3NF        | 1NF           | composite primary key | data integrity        | data integrity |  |
| redundancy            | redundancy | foreign keys  |                       |                       |                |  |

Normalization is a database design technique used to reduce

**[redundancy]** and improve **[data integrity]**.

The first normal form ( **[1NF]** ) requires that all columns contain **[atomic]** values.

Second normal form ( **[2NF]** ) addresses **[partial dependencies]** in tables with a **[composite primary key]**.

Third normal form ( **[3NF]** ) eliminates **[transitive dependency]**, where a non-key attribute depends on another non-key attribute.

To establish **[relationships]** between tables, relational databases use **[foreign keys]**.

A table that models a [ ] **[many-to-many]** relationship often requires a linking table like [ ] **[OrderItems]** with a composite primary key.

In an e-commerce database, the [ ] **[Orders]** table would typically have a foreign key referencing the [ ] **[Customers]** table.

In a normalized library database, author information (like nationality) is stored in a [ ] **[separate]** table to avoid [ ] **[redundancy]** .

In a school management system, the combination of StudentID and CourseID would form a [ ] **[composite primary key]** in an enrollment table.

Normalization helps to ensure [ ] **[data integrity]** by eliminating redundant data and enforcing data dependencies.

---

*All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.*

---

 **Report a problem**